One-Way Door

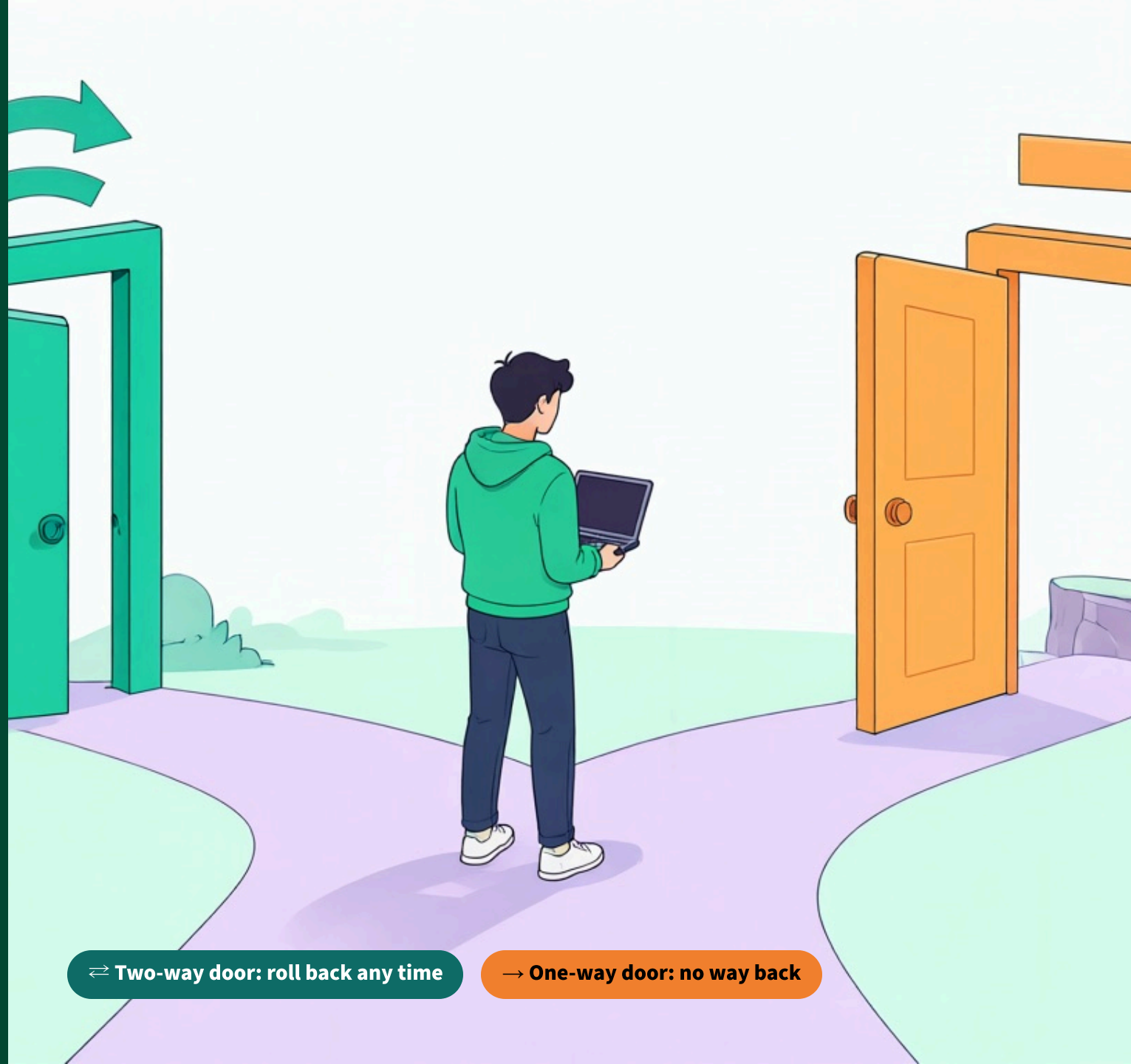
HACKATHON PROPOSAL · V2

Know before you merge whether you can roll back.

A GitHub Copilot skill that reviews every pull request for rollback safety. Told through Northwind Payments, and the Friday afternoon a sensible refactor booked 50 settlements at £0.00.



October 2026



THE QUESTION NO PIPELINE ASKS

“If this goes wrong after deploy, can we actually roll back?”

Tests, CI and code review check new code against new data. Nobody checks whether the **old** code can still run against the data the new code leaves behind.

⇄ Two-way door

Ship it, roll it back, and the old code still works.

- a new nullable column
- a new optional field
- a new endpoint

→ One-way door

It leaves state behind that the old code can't handle. Rollback doesn't restore service. Often it makes things worse.

- migrated tables
- new-format events on a topic
- rewritten cache entries
- payloads sent to partners



I ask this on every pull request.

1 pull request

2 offline scripts

3 Java services

50 rows at £0.00

4 safe PRs instead

MEET NORTHWIND PAYMENTS

The team, and the Friday they'll remember

Northwind settles card payments for small UK merchants. Every settlement becomes an event, then a ledger row, then part of finance's daily total.



Priya

Backend developer · raises PR #482

Money as a double? Let's store pence, properly.



Tom

Tech lead · reviews PR #482

Tests are green. Approved.



Sam

On call this Friday

If it goes wrong, I'll roll it back.



Grace

Finance · runs the reconciliation

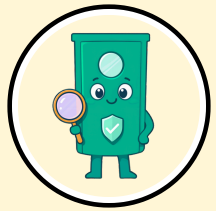
The ledger must match the bank, to the penny.



Dev

Database administrator

Who renamed my column?



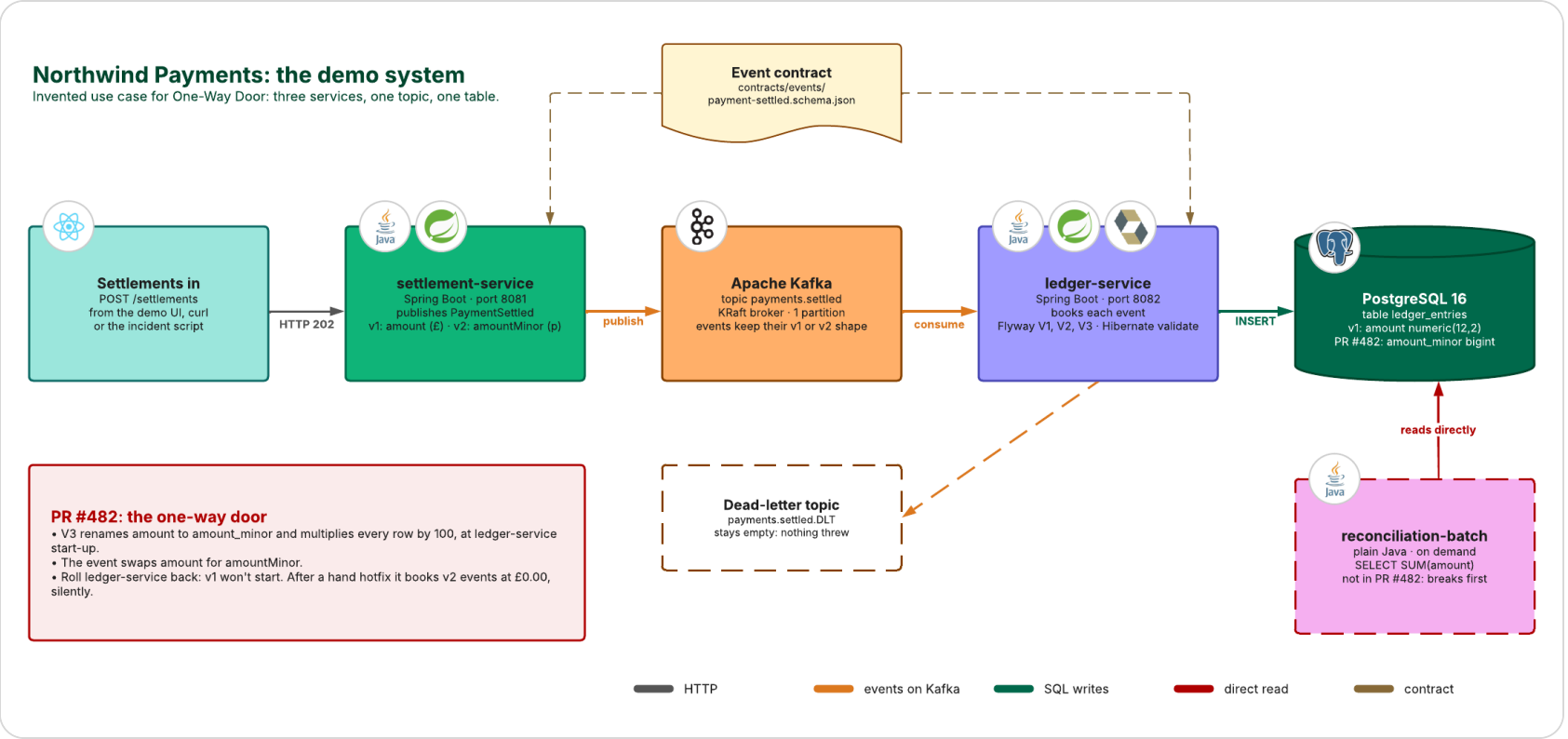
Hinge

The One-Way Door skill

Before you merge: can you roll back?

THE SYSTEM

Three services, one topic, one table



The writer

settlement-service turns each settlement into a `PaymentSettled` event on Kafka.

The reader

ledger-service books one row per event in `ledger_entries`, and owns that table.

The reader nobody remembered

reconciliation-batch, finance's job, sums the table directly.

Drawn in Lucidchart. All three services are real Java code, with their own tests and git history.

THE THREE SERVICES, BEFORE PR #482 (V1, ON MAIN)

How a settlement becomes money in the ledger

1 · The writer

settlement-service

Spring Boot · Java 21 · :8081. Takes `POST /settlements` and publishes `PaymentSettled`. It waits for Kafka's acknowledgement, so a 202 means the event is really there.

```
record PaymentSettled(
  UUID paymentId,
  String merchantId,
  String currency,
  double amount,
  Instant settledAt)
```

The detail that matters: no Java type header. Readers map the JSON onto their own class by field name.

2 · The reader

ledger-service

:8082 · Flyway · Hibernate. Consumes each event, skips duplicates, inserts one row. Flyway migrates at start-up; Hibernate checks the table matches the code.

```
@KafkaListener(
  topics = "payments.settled")
void on(PaymentSettled e) {
  entries.save(
    LedgerEntry.from(e));
} // amount numeric(12,2)
```

Events that throw go to `payments.settled.DLT` after two retries.

3 · Nobody remembered it

reconciliation-batch

Plain Java · JDBC · on demand. Finance's daily total, compared with the bank statement. It reads Postgres directly, not through ledger-service.

```
SELECT COUNT(*),
  COALESCE(SUM(amount), 0)
FROM ledger_entries;
```



Owned by Grace's finance team. Not in PR #482. No tests.

WHY PR #482 WAS RAISED

Why swap a `double` for whole pence?



A double can't hold 0.10 exactly. In a payments ledger, that's a bug waiting to happen. Let's store pence as a whole number, like the card schemes do.

1 Doubles are binary fractions

```
0.1 + 0.2
== 0.30000000000000004
```

Most decimal amounts can't be stored exactly. Each one is very slightly wrong.

2 Small errors add up

Sum millions of settlements and totals can drift by pennies. Services round in different places. Finance must explain every penny.

3 Whole pence can't round

```
// before
double amount = 268.47;
// after
long amountMinor = 26847;
```

Card schemes send amounts in minor units. An integer is exact.

The idea is right. The trouble is how it ships: the table, the event and both services, all in one deploy.

PR #482 · BRANCH PR-482-MINOR-UNITS

One tidy pull request. Four changes at once.

1 Flyway V3 renames the column

```
ALTER TABLE ledger_entries
  RENAME COLUMN amount
  TO amount_minor;
```

2 ...and rewrites every row in pence

```
ALTER COLUMN amount_minor
  TYPE bigint USING
  (amount_minor * 100)::bigint;
```

£268.47 becomes 26847. The old values are gone.

3 The event swaps its field

```
PaymentSettled {
  - double amount
  + long amountMinor
}
```

4 Both services change together

settlement-service sends `amountMinor`. ledger-service reads it and writes `amount_minor`. One deploy.



Tom
reviewer

```
✓ unit tests: all green
✓ builds
✓ 9 files changed
```

Approved

Not in the PR

reconciliation-batch, which still reads `amount`. And no plan for events already on Kafka.

PR #482 · THE REVIEW

Tom's review was careful. It just didn't ask about yesterday.



What was checked

- ✅ It compiles
- ✅ Unit tests pass, on both services
- ✅ The migration runs on a clean database
- ✅ Both services agree on the new event
- ✅ The code reads well

Looks good to me.



What nobody asked

- ❓ Can the **previous** release still run after V3?
- ❓ Who else reads `ledger_entries.amount`?
- ❓ What about the events already on `payments.settled`?
- ❓ If only ledger-service rolls back, what does **settlement-service** keep sending?
- ❓ How long after deploy is a rollback still safe?

These are my questions. I ask them on every PR.

Every unasked question on the right is a moment that broke on Friday: **16:55, 16:58, 17:00 and 17:26.**

PR #482 · FLYWAY AND V3

The migration that closes the door in 5 milliseconds

```
db/migration/V3__amount_minor_units.sql (lines 1-2 are comments)
3  ALTER TABLE ledger_entries
     RENAME COLUMN amount TO amount_minor;
4  ALTER TABLE ledger_entries
     ALTER COLUMN amount_minor TYPE bigint
     USING (amount_minor * 100)::bigint;
```

Line 3: rename

From this moment nothing called `amount` exists. Old ledger code and the reconciliation break.

Line 4: rewrite in place

Every past row becomes pence. Only a script that divides by exactly 100 can bring it back.

In the recorded run, Flyway applied V3 in **0.005 seconds**, before ledger-service took its first event. That's the door closing.

What happens when ledger-service starts

1. Flyway lists its `V…__….sql` files
2. It reads `flyway_schema_history`
3. It runs what's new, each in a transaction
4. Hibernate checks the table matches the code
5. Only then does the Kafka listener start

It only goes forward. No undo in the free edition, and none can un-mix events on Kafka.

Expand first. The safe route's V3 just adds `amount_minor`, nullable.

THE INCIDENT · ACT 2 OF THE DEMO · FRIDAY 16:30 TO 16:58

Friday afternoon at Northwind, minute by minute

16:30



v1 runs. Booked in pounds.

settlement-service publishes `amount: 268.47`. ledger-service books it in `numeric(12,2)`. Ten settlements, all correct.

```
10 rows · £0.00 rows: 0
```

Last change before the weekend.

16:40



PR #482 ships. Tests were green.

Flyway runs V3 before the first event: every row rewritten in pence. settlement-service v2 now sends `amountMinor: 26847`, and no `amount`.

Green. Ship it.

16:55



The reconciliation breaks.

Nobody touched reconciliation-batch. It still runs `SUM(amount)`.

```
ERROR: column "amount" does not exist
```

Where did my column go?

16:58



Roll back the ledger. It won't start.

Sam rolls back **ledger-service only**. The old code meets the new table. Hibernate stops it. Flyway can't undo.

```
Schema-validation: missing column [amount] in table [ledger_entries]
```

Why won't it start?

THE INCIDENT · FRIDAY 17:00 TO 17:26

The fix that made it quiet, not right

17:00



Settlements keep coming.

settlement-service is still v2. Nobody rolled it back. Fifty `amountMinor` events queue on `payments.settled`, with no ledger to read them.

```
backlog: 50 events · ledger-service: FAILED
```

Why is the backlog growing?

17:25



A hand-written hotfix.

`hotfix-reverse-v3.sql` renames the column back and divides by 100. Hibernate is happy. v1 starts and drains the fifty waiting events.

```
ledger-service: UP
```

Reversed by hand. Under pressure.

17:26



Every dashboard is green.



Fifty settlements booked at **£0.00**. No exception. Nothing in the dead-letter topic. ledger-service healthy.

17:26 · THE QUIET FAILURE

£12,708.11

missing, and nothing errored

Settlements sent	£19,358.63
Booked in the ledger	£6,650.52
Booked at £0.00	50 settlements
Exceptions	0
Dead-letter topic	0 records
ledger-service	UP



The bank says one number.
The ledger says another.
Nothing told us.

Figures from a recorded run of the demo.



THE INCIDENT · WHAT WENT WRONG

Everyone watched the ledger. Nobody thought about settlement-service.

The plan was “roll back the service that broke”. But PR #482 changed two services, and the writer kept writing.

settlement-service
still v2
never rolled back

→

payments.settled
`{"amountMinor": 26847}`
50 events, no `amount`

→

ledger-service
v1, rolled back
expects `amount`

→

£0.00
× 50

1 The writer kept writing

settlement-service was never rolled back. From 16:58 every new settlement arrived in the new shape, and the old reader couldn't read it.

2 A topic is state, like a table

Rolling back settlement-service too wouldn't help: the events already on Kafka stay v2. Rolling back code doesn't roll back data.

3 Old and new overlap

In every deploy and rollback, old and new versions run side by side. The writer and reader must understand each other in every mix, not just at the end.



I rolled back the ledger. The settlements already sent didn't roll back with it.

THE INCIDENT · WHY £0.00 AND NOT AN ERROR

A v2 event meets v1 code

The v2 event on payments.settled

```
{
  "paymentId": "1d1c5a8e-…",
  "merchantId": "merchant-0026",
  "currency": "GBP",
  "amountMinor": 26847,   ← ignored
  "settledAt": "17:12:30Z"
}
```

→

What v1 ledger-service binds it to

```
record PaymentSettled(
  UUID paymentId,        ← "1d1c5a8e…"
  String merchantId,     ← "merchant-0026"
  String currency,       ← "GBP"
  double amount,         ← missing: 0.0
  Instant settledAt)     ← 17:12:30Z
```

→

ledger_entries row 80

£0.00

should be £268.47

It reads by field name. No type header (`use.type.headers: false`), so any JSON maps onto v1's class.

Unknown fields are ignored. Jackson's default. `amountMinor` is dropped without a word.

A missing double is 0.0. A primitive keeps its default, and `BigDecimal.valueOf(0.0)` books £0.00.

Nothing reaches the DLT. Only records that throw go there. Nothing threw.

The loud failure came first. `ddl-auto=validate` refused the renamed table. The £0.00 is the quiet one.

There's no undo. V3 rewrote every value in place. Going back meant a hand-written script.



Ask me before you merge. I'll tell you which kind of door it is.

THE SKILL

One-Way Door: a GitHub Copilot skill for rollback safety

One folder in `.github/skills/one-way-door`. Copilot loads it when someone reviews a PR, checks a release, changes a migration or an event, or asks “can we roll back?”.

1 Two-way or one-way

Classifies every change: safe to roll back, or not.

2 When the door closes

For each one-way change: when it closes, what breaks on rollback, and the safe rollback window.

3 The blast radius

Other code that reads the same table, column, topic or field, even outside the PR.

4 A rollback-safe split

Expand → migrate → contract → drop. It can generate the PRs.

VS Code agent mode **Copilot CLI** **Copilot cloud agent** **Copilot code review**

THE SKILL · HOW IT WORKS

Scripts check. Copilot judges.

1 The PR goes up
#482 against main

2 Someone asks
“can we roll back?”

3 The skill loads
its description matches

4 Scripts check
offline, JSON out

5 Copilot judges
readers, old code

6 The verdict
as a PR comment

What's in the folder

`SKILL.md`	The instructions. Its description is what makes Copilot load it.
`patterns.md`	24 one-way patterns, each with its safe split.
`output-format.md`	The exact shape of the PR comment.
`scan-migrations.mjs`	Destructive DDL in Flyway, Liquibase and EF migrations.
`schema-compat.mjs`	Event schema diff, checked from the old reader's side.

Around it

- A path rule for migrations and event contracts
- A hook that stops Copilot editing migrations already on main
- A plugin for one-step install

Built to be trusted

offline · Node 20+ standard library · JSON out · always exit 0 · never changes code unasked

THE SKILL · THE REVIEW · ACT 3 OF THE DEMO

What Copilot says before anyone merges

GitHub Copilot commented on PR #482 · used `.github/skills/one-way-door`

One-way door check

Verdict: ONE-WAY · Safe rollback window: **none** (closes when V3 runs at ledger-service start-up)

1. **Column renamed and converted to pence in place.** If rolled back: v1 fails start-up on schema validation.
2. **Event field `amount` replaced by `amountMinor`.** Rolling back doesn't remove events already on the topic.
3. **A missing `amount` silently becomes £0.00.** Nothing reaches the DLT.

Blast radius: reader	Uses	Breaks when
reconciliation-batch (not in this PR)	`SUM(amount)`	V3 runs
ledger-service	`amount` / `amount_minor`	rollback
ledger-service consumer	`PaymentSettled.amount`	rollback with a v2 backlog

Suggested split: expand → migrate → contract → drop. Reply “generate the split”.
scan-migrations: 3 findings · schema-compat: 5 findings · checked main...pr-482-minor-units

It predicted the incident



PREDICTED · seen at 16:58

The old ledger won’t start

```
Schema-validation: missing column [amount]
```



PREDICTED · seen at 16:55

reconciliation-batch breaks, though it isn’t in the PR

```
column "amount" does not exist
```



PREDICTED · seen at 17:26

Settlements booked at £0.00, silently

```
50 rows at £0.00 · DLT 0 · ledger UP
```

THE SAFE ROUTE · ACT 4 OF THE DEMO

Same destination. Four deploys. Each one reversible.

The split ends exactly where PR #482 does: the same table, the same event. It keeps old and new side by side until nobody needs the old one.

1 Expand

add the new

Table: V3 adds `amount_minor`, nullable. The ledger writes both.

Event: settlement-service sends `amount` and `amountMinor`.

Readers: reconciliation unchanged: `SUM(amount)`.

⇄ Roll back: old code ignores the new column and field.

2 Migrate

copy the old data

Table: V4 backfills `amount_minor` and checks every row agrees.

Event: still sends both.

Readers: reconciliation also reports any mismatch.

⇄ Roll back: both columns are there and correct.

3 Contract

switch the readers

Table: V5: the ledger reads only `amountMinor`; a trigger keeps `amount` filled.

Event: still sends both. No `amount`? To the DLT, not £0.00.

Readers: reconciliation sums `amount_minor`.

⇄ Roll back: `amount` is still kept up to date.

4 Drop

remove the old

Table: V6 drops `amount` and the trigger.

Event: settlement-service sends `amountMinor` only.

Readers: nobody reads `amount` any more.

⇄ Safe because nothing uses the old any more.

PR #482: one deploy, and no way back.

The split: four deploys, with a way back at every step.

THE PROOF · ACT 5 OF THE DEMO

Every step rolls back cleanly. PR #482 doesn't.

old → new	1 · baseline	2 · deploy ledger	3 · deploy settlement	4 · roll back ledger	5 · roll back settlement	result
main → split 1	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
split 1 → split 2	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
split 2 → split 3	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
split 3 → split 4	✓ 20/20	✓ 40/40	✓ 60/60	✓ 80/80	✓ 100/100	PASS
main → PR #482	✓ 20/20	✗ 20 at £0	✗ 20 at £0	✗ 20 at £0	✗ 20 at £0	FAIL

PR #482 fails at stage 2, before anyone presses rollback: the new ledger reading old events books zero too.

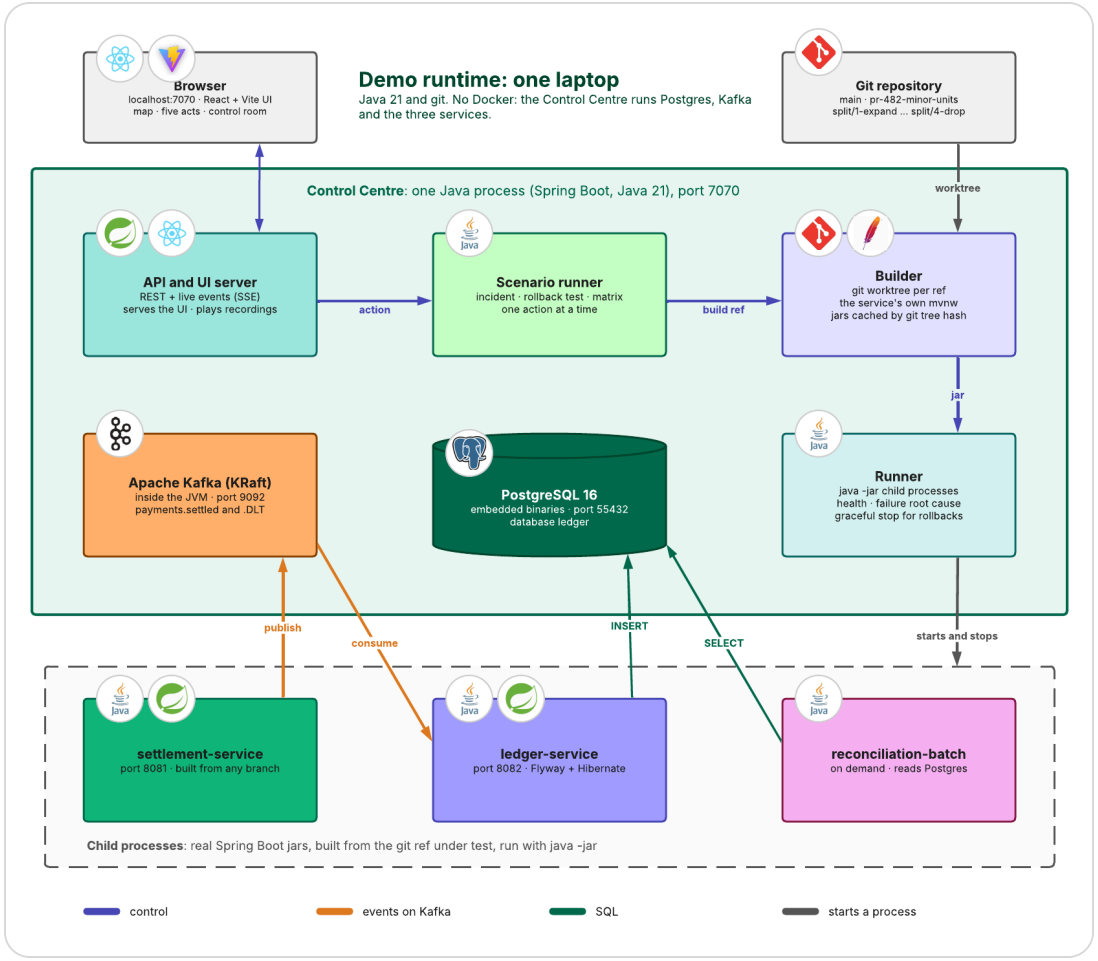
Each row, for real

A fresh database on the old version, then five stages. After each, 20 settlements and six checks:

- **both services healthy**
- **every settlement booked once**
- **nothing at £0.00**
- **DLT empty**
- **no backlog**
- **reconciliation = money sent**

Why five stages?

A real deploy is never instant: old and new run side by side (stages 2 and 4). A real rollback only rolls back code: the database keeps the new migration (stages 4 and 5). A step passes only if every mix works.



THE DEMO

The whole story, live

No Docker. One Java process starts a real Postgres and a real Kafka, builds each service from its git branch, and runs them side by side. The browser shows it all.

Act	What you see	Time
1 · The PR	The V3 migration; tests go green	0:35
2 · The incident	Seven steps, ending at £0.00	1:10
3 · The review	Copilot's verdict beside the real errors	1:00
4 · The split	Expand, migrate, contract, drop	0:35
5 · The proof	The rollback matrix, run live	0:40

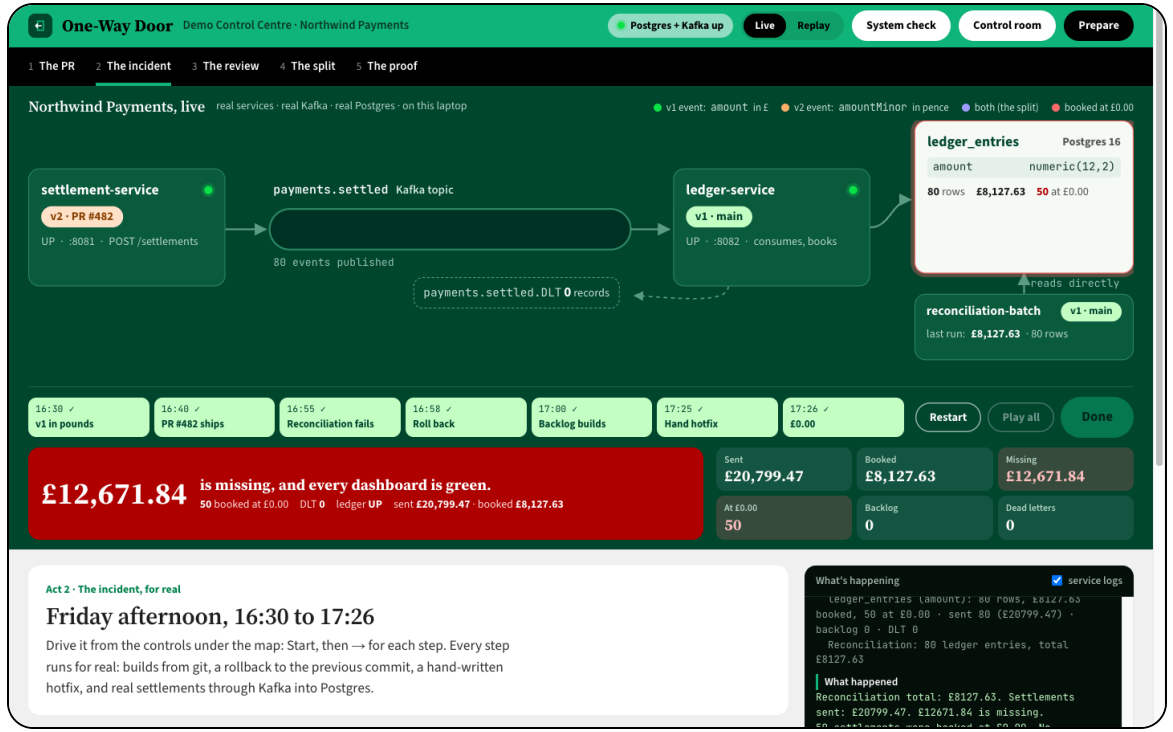
Needs Java 21+ and git. Tested on Windows and macOS.

```
demo/start.sh        # macOS
demo\start.cmd       # Windows
```

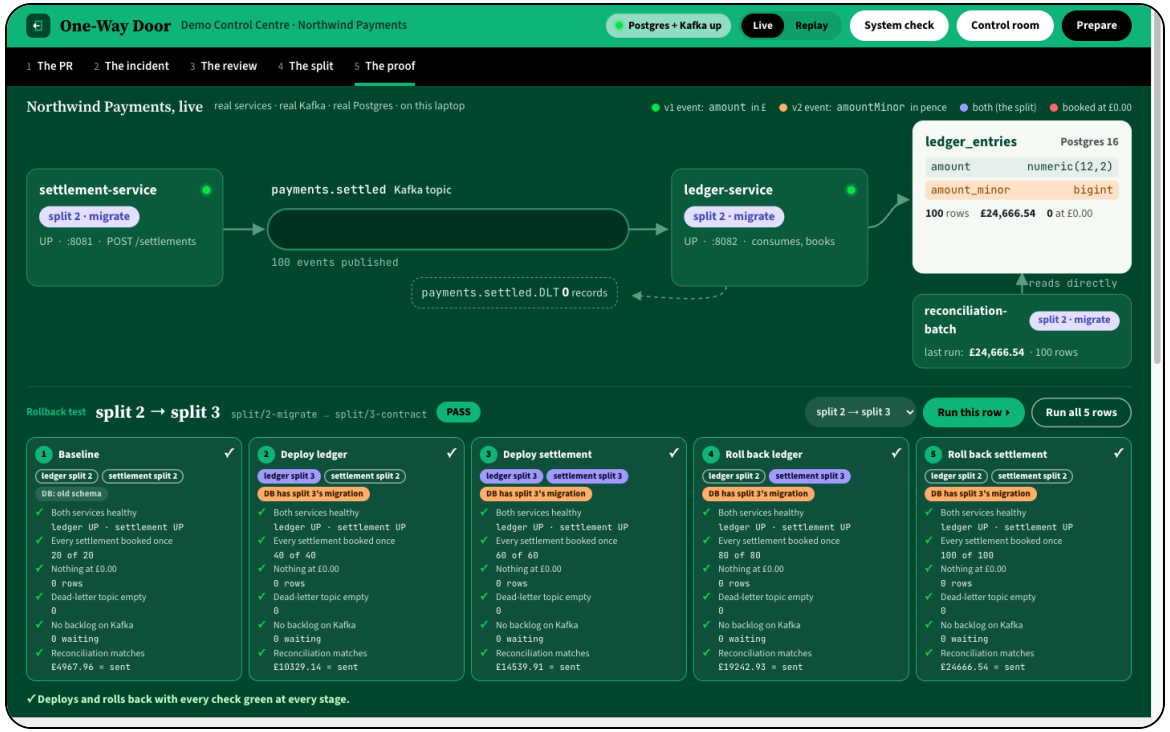
Replay mode plays a recorded real run if the venue laptop misbehaves.

THE DEMO · THE CONTROL CENTRE, LIVE

What the audience sees



Act 2 · 17:26. A live run: the map, the seven steps under it, and £12,671.84 missing with every service UP. Amounts are random each run.



Act 5 · the proof. One row run live: deploy ledger, deploy settlement, roll each back. Six checks after every stage. PASS in about 20 seconds.

THE DEMO · UNDER THE HOOD

One click, traced: “Roll back” at 16:58

1. You press **Run step 4**. The browser sends `POST /api/incident/next`.
2. The Control Centre queues it on one scenario thread, so two clicks can never race.
3. The builder finds main's ledger-service jar in its cache, built once from a git worktree.
4. The runner stops ledger-service v2 through `/actuator/shutdown`. Its Kafka consumer leaves cleanly.
5. It starts the v1 jar on the same Postgres and Kafka. Hibernate validates: **APPLICATION FAILED TO START**.
6. The runner reads the root cause from the log and streams it to the browser. The map turns ledger-service red.



It's not a mock. The rollback really runs the previous commit's code against today's data.

No faking: every rollback builds the previous git ref and runs it against the current database and topic.

WHERE IT PLUGS IN

One folder, every Copilot surface

VS Code

Agent mode

Ask before you push: “can we roll back main...my-branch?”

Terminal

Copilot CLI

The same skill, or `/one-way-door`.

GitHub

Cloud agent

Reviews the PR on GitHub, using the folder in the repo.

GitHub

Code review

Copilot code review can use repository skills on every PR.

Guardrails: what it will and won't do

✓ Reviews. It never changes code unless asked.

✓ Offline: git and Node only. No servers, accounts or network calls.

✓ Scripts always exit 0. Findings are data, not errors.

✓ The hook only blocks edits to migrations already on main.

✓ The hook fails open. A broken hook never blocks your work.

✓ Readers it can't see are assumed to exist.

BEFORE AND AFTER

Same question, same PR. Two very different answers.

“Review pr-482-minor-units against main. If this goes wrong after deploy, can we roll back?”

Without the skill · illustrative

“This looks like a clean refactor. Consider adding a down migration. To roll back, revert the commit and redeploy.”

- No mention of reconciliation-batch
- No mention of events already on Kafka
- “Revert and redeploy” is exactly what failed at 16:58

With the skill · from the worked example

ONE-WAY. No safe window. The old ledger won't start; reconciliation-batch breaks; a v2 backlog books £0.00 silently. Split it into four PRs.

- Names the reader outside the PR
- Explains the £0.00 before it happens
- Offers a route that keeps a way back

One-Way Door

Every merge is a door.
Know which kind before you walk through it.

Copy one folder into a repo, or install it once as a Copilot plugin. It needs git and Node 20+. No servers, no accounts, no network calls.

```
cp -R .github/skills/one-way-door your-repo/.github/skills/
```





Northwind Payments, its people and its amounts are invented for this demo.
Figures come from a recorded run of the demo Control Centre.